

The Complete FastAPI Guide

From first endpoint to production-grade microservices

4 PHASES

35+ TOPICS

4 PROJECTS

2026 EDITION

Built on the official FastAPI documentation at fastapi.tiangolo.com

Covers FastAPI 0.124+ • Pydantic v2 • Python 3.10+

Preface

FastAPI is a modern, high-performance web framework for building APIs with Python, designed around standard Python type hints. It rests on two well-engineered foundations: **Starlette** for the ASGI web layer and **Pydantic** for data validation and serialization. The result is a framework that gives you the speed of Node.js or Go, the developer ergonomics of a typed language, and automatic, always-correct OpenAPI documentation — for free.

This guide was written for the developer who wants one document, end-to-end, that takes them from `pip install fastapi` to deploying versioned microservices behind a load balancer. Every concept is explained with working code; every phase ends with a small, self-contained project that exercises the ideas you just learned. If you finish this guide, you will not need to consult Stack Overflow to ship a production FastAPI service.

How to use this guide

Read the phases in order. Each one assumes the previous. Type the code by hand — do not copy and paste. The framework rewards muscle memory of its decorators and patterns, and you will retain ten times more by typing.

- **Phase 1 — Beginner.** The mental model: path operations, parameters, request bodies, Pydantic models, automatic docs.
- **Phase 2 — Intermediate.** Structure and integration: dependency injection, routers, async, databases, auth, middleware.
- **Phase 3 — Advanced.** The serious craft: WebSockets, lifespan, caching, queues, testing, observability.
- **Phase 4 — Professional.** Production: Docker, security hardening, versioning, SDK generation, microservices, CI/CD.

***Note.** This guide reflects FastAPI as of the 2026 release line (0.124+). Where the API has changed historically — for example, the move from event handlers to lifespan, or from Pydantic v1 to v2 — only the current recommendation is shown.*

Prerequisites

- Python 3.10 or newer installed (3.12 or 3.13 recommended in 2026).
- Comfort with the command line and a code editor.
- Basic understanding of HTTP: methods, status codes, JSON.
- Familiarity with Python type hints (`list[int]`, `str` | `None`, etc.).

Table of Contents

Preface

Introduction: Why FastAPI

Phase 1 — Beginner

1.1 Setup & Installation

1.2 Path Operations (GET, POST, PUT, DELETE)

1.3 Path Parameters

1.4 Query Parameters

1.5 Request Body

1.6 Pydantic Models — Validation, Types, Fields

1.7 Response Models & Status Codes

1.8 Forms & File Uploads

1.9 Automatic Docs — Swagger UI & ReDoc

★ Phase 1 Project — Personal Bookshelf API

Phase 2 — Intermediate

2.1 Dependency Injection (Depends)

2.2 Routers & Project Structure (APIRouter)

2.3 Async / Await in FastAPI

2.4 Databases with SQLAlchemy (sync)

2.5 Async Databases

2.6 Alembic Migrations

2.7 Authentication — OAuth2 & JWT

2.8 Middleware — CORS & Custom

2.9 Error Handling & Custom Exception Handlers

2.10 Background Tasks

★ Phase 2 Project — Task Manager API

Phase 3 — Advanced

3.1 Advanced Pydantic v2 — Validators & Computed Fields

3.2 WebSockets

3.3 Lifespan & Startup/Shutdown Events

3.4 Rate Limiting with slowapi

3.5 Caching & Redis

3.6 Task Queues with Celery

3.7 Logging & Structured Logs

3.8 Testing — pytest, TestClient, Mocking

3.9 Monitoring — Prometheus & Sentry

★ Phase 3 Project — Real-time Chat & Notifications

Phase 4 — Professional

4.1 Docker & Deployment (gunicorn + uvicorn)

4.2 Security Best Practices — HTTPS, Secrets, OWASP

4.3 API Versioning & Design

4.4 OpenAPI Schema & SDK Generation

4.5 Performance Tuning

4.6 Microservices with FastAPI

4.7 CI/CD for FastAPI Apps

★ Phase 4 Project — Production-ready URL Shortener

Appendix A — HTTP Status Code Reference

Appendix B — Recommended Project Layout

Appendix C — Cheat Sheet

Introduction: Why FastAPI

Before we write a single line of code, it is worth understanding why FastAPI exists and what makes it different. In the Python web ecosystem, Django and Flask each occupied a clear niche for over a decade: Django for batteries-included full-stack applications, Flask for minimal micro-services. Both were built on WSGI, the synchronous Web Server Gateway Interface, which means a worker process handles one request at a time. When the request blocks on the database or a network call, the worker waits.

FastAPI was released in December 2018 by Sebastián Ramírez to address three problems at once: **(1)** manual request validation in Flask led to runtime bugs that type-aware tooling could catch at the boundary; **(2)** WSGI's synchronous model left modern CPUs idle while waiting on I/O; **(3)** documentation drifted from the code the moment either changed. FastAPI's answer is a single, elegant idea: *your type hints are the source of truth*. From them, the framework derives validation, serialization, OpenAPI schema, Swagger UI, ReDoc, and client SDK generation.

The three pillars

- **Starlette** — the ASGI toolkit underneath. It provides routing, middleware, WebSockets, background tasks, the test client, and the async event loop integration.
- **Pydantic v2** — the data layer. Compiled in Rust as of v2, it validates, coerces, and serializes data based on Python type annotations. Roughly 5–50× faster than v1.
- **Python type hints** — the developer interface. FastAPI inspects function signatures and Pydantic models to wire everything together automatically.

What you get out of the box

- Automatic request validation with detailed JSON error responses.
- Automatic serialization of responses (objects, dataclasses, ORM models).
- An OpenAPI 3.1 schema generated at runtime, no annotations needed.
- Interactive API docs at `/docs` (Swagger UI) and `/redoc` (ReDoc).
- Async/await first-class — endpoints can be coroutines.
- Dependency injection that composes naturally with type hints.
- Security utilities for OAuth2, JWT, API keys, HTTP Basic, and more.
- WebSocket support, background tasks, file streaming, server-sent events.

Performance, briefly

FastAPI applications routinely handle 3,000–20,000 requests per second on commodity hardware, depending on workload. The framework itself adds very little overhead above Starlette; Pydantic v2's Rust core handles validation at near-native speed. Most performance work in a FastAPI service is database tuning, not framework tuning.

PHASE 1

Beginner

The first endpoint, the type system, and the auto-generated docs that make FastAPI feel like magic.

1.1 Setup & Installation

FastAPI runs on any modern Python. In 2026 the recommended target is Python 3.12 or 3.13; the minimum supported is 3.10. We will create an isolated virtual environment so the dependencies of this project do not pollute the system Python.

Create a virtual environment

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate

# Windows (PowerShell)
py -m venv .venv
.venv\Scripts\Activate.ps1
```

Install FastAPI

The official guidance is to install the *standard* extra. It pulls in FastAPI itself plus the conveniences you almost always want during development: the Uvicorn server, Jinja2 for templating, python-multipart for form parsing, and the FastAPI CLI.

```
pip install "fastapi[standard]"
```

Note. The quotes around `fastapi[standard]` matter. Many shells — *zsh* in particular — interpret square brackets as glob patterns and will fail without quoting.

Your first application

Create a file called *main.py*:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"hello": "world"}
```

Run the development server

```
fastapi dev main.py
```

Open `http://127.0.0.1:8000` in your browser. You should see the JSON response. Now visit `http://127.0.0.1:8000/docs` — FastAPI has already built a complete interactive Swagger UI for the one endpoint you wrote. This is the moment most people fall in love with the framework.

Tip. `fastapi dev` enables auto-reload on file changes. For production, use `fastapi run` (no reload) or invoke Uvicorn directly with your chosen worker configuration — we will cover this in Phase 4.

1.2 Path Operations (GET, POST, PUT, DELETE)

In FastAPI's vocabulary a *path operation* is the combination of an HTTP method (GET, POST, ...) and a URL path. You declare one by decorating a function with `@app.get("/path")`, `@app.post(...)`, and so on. The function below the decorator runs whenever a matching request arrives.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items")
def list_items():
    return [{"id": 1, "name": "Pen"}, {"id": 2, "name": "Pad"}]

@app.post("/items")
def create_item(item: dict):
    return {"created": item}

@app.put("/items/{item_id}")
def replace_item(item_id: int, item: dict):
    return {"id": item_id, "replaced_with": item}

@app.delete("/items/{item_id}")
def delete_item(item_id: int):
    return {"deleted": item_id}
```

Choosing the right verb

- **GET** — retrieve a resource. Idempotent and safe; never use it to mutate state.
- **POST** — create a resource, or invoke an action whose effects are non-idempotent.
- **PUT** — replace a resource in full. Idempotent: repeated calls produce the same state.
- **PATCH** — partial update. Often more practical than PUT for real APIs.
- **DELETE** — remove a resource. Idempotent: deleting twice is the same as deleting once.

How path matching works

FastAPI evaluates routes in the order they are declared. If two routes could match the same URL, the first one wins. This matters when you have both a static route and a parameterised one — declare the static route first.

```
# Correct ordering
@app.get("/users/me")
def current_user():
    return {"username": "alice"}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```


1.3 Path Parameters

Anything in curly braces inside the URL is a *path parameter*. FastAPI passes it to your function as an argument of the same name. The type annotation on the argument tells FastAPI how to parse and validate it.

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Visit `/items/5` and FastAPI returns `{"item_id": 5}`. Visit `/items/foo` and you get a clean 422 Unprocessable Entity response with a structured error explaining that `foo` is not a valid integer. You did not write a single line of validation code.

Constrained path parameters

Use *Path* with *Annotated* to attach constraints — minimum, maximum, regex, descriptions for the OpenAPI docs.

```
from typing import Annotated
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(
    item_id: Annotated[int, Path(ge=1, le=10_000, description="The item ID")]
):
    return {"item_id": item_id}
```

Enum path parameters

When the set of valid values is small and known, an *Enum* gives you a drop-down in the docs and rejects everything else.

```
from enum import Enum
from fastapi import FastAPI

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

app = FastAPI()

@app.get("/models/{model_name}")
def get_model(model_name: ModelName):
    return {"model": model_name.value}
```

1.4 Query Parameters

Any function argument that is *not* in the path and is not a Pydantic model becomes a query parameter. If it has a default value, it is optional; if not, it is required.

```
from fastapi import FastAPI

app = FastAPI()

items = [{"name": f"Item {i}"} for i in range(100)]
```

```
@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return items[skip : skip + limit]
```

A request to `/items?skip=20&limit=5` returns five items starting from the twentieth. The values arrive as strings on the wire; FastAPI coerces them to integers and rejects anything that does not fit.

Optional and nullable parameters

```
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/search")
def search(
    q: Annotated[str | None, Query(max_length=50, min_length=2)] = None,
    sort: Annotated[str, Query(pattern="^(asc|desc)$")] = "asc",
):
    return {"q": q, "sort": sort}
```

List query parameters

Repeated query keys collect into a list:

```
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/tags")
def by_tags(tag: Annotated[list[str], Query()] = []):
    return {"tags": tag}
```

Request `/tags?tag=python&tag=fastapi` returns both values in a list.

1.5 Request Body

To accept structured data in the body of a request, declare a Pydantic model and use it as a function argument. FastAPI reads the body as JSON, validates it against the model, and passes you a fully-typed Python object.

```
from fastapi import FastAPI
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None

app = FastAPI()

@app.post("/items")
def create_item(item: Item):
    return {"name": item.name, "total": item.price * 1.2}
```

Send a POST to `/items` with a JSON body like `{"name": "Pen", "price": 1.50}`. FastAPI parses, validates, and gives you `item` as an `Item` instance. Try sending `{"name": "Pen", "price": "not a number"}` and the response is a structured 422 explaining the failure — again, no validation code on your part.

Mixing path, query, and body

FastAPI happily combines all three in a single endpoint. It looks at each argument and decides where to get it from based on its type and whether it appears in the path.

```
@app.put("/items/{item_id}")
def update_item(
    item_id: int,          # path parameter
    item: Item,            # body (Pydantic model)
    notify: bool = False,  # query parameter
):
    return {"id": item_id, "item": item, "notify": notify}
```

1.6 Pydantic Models — Validation, Types, Fields

Pydantic is the layer that makes FastAPI's magic possible. A Pydantic model is a class that inherits from *BaseModel* and declares its attributes with type hints. Pydantic enforces those types, runs any validators you attach, and emits a JSON Schema describing the shape — which FastAPI then surfaces in the OpenAPI docs.

Built-in types

Pydantic handles all standard Python primitives — *int*, *float*, *str*, *bool*, *bytes* — and collection types (*list[T]*, *dict[K,V]*, *set[T]*, *tuple[...]*), plus several useful specialised types out of the box.

```
from datetime import datetime, date, time, timedelta
from uuid import UUID
from pydantic import BaseModel, EmailStr, HttpUrl, IPvAnyAddress

class Profile(BaseModel):
    id: UUID
    email: EmailStr          # validates RFC-5322 email syntax
    website: HttpUrl         # must parse as a valid URL
    last_login: datetime
    birth_date: date
    ip: IPvAnyAddress
    session_length: timedelta
```

Note. *EmailStr* requires the *email-validator* package. Install it with `pip install "pydantic[email]"`.

Field constraints with Field()

Use *Field* to attach metadata: defaults, descriptions, examples, and constraints. Everything you write here also shows up in the Swagger UI.

```
from pydantic import BaseModel, Field

class Product(BaseModel):
    name: str = Field(min_length=1, max_length=120,
                      description="Human-readable product name")
    price: float = Field(gt=0, le=1_000_000, examples=[19.99])
    sku: str = Field(pattern=r"^[A-Z0-9]{6,12}$")
    tags: list[str] = Field(default_factory=list, max_length=10)
    stock: int = Field(default=0, ge=0)
```

Nested models

Models compose. You can nest one inside another and FastAPI will validate the full tree on input and serialise it on output.

```
from pydantic import BaseModel

class Address(BaseModel):
    street: str
    city: str
    postcode: str

class Customer(BaseModel):
    name: str
    address: Address
    other_addresses: list[Address] = []
```

Type coercion

By default Pydantic v2 will coerce sensibly: an integer-shaped string into an *int*, ISO-8601 strings into *datetime*, and so on. If you want strict mode (no coercion) on a particular field, use *StrictInt*, *StrictStr*, or set *model_config = ConfigDict(strict=True)* on the model.

1.7 Response Models & Status Codes

When you return a value from a path operation, FastAPI serialises it to JSON. You almost always want to be explicit about the *shape* of that response — for documentation, for client-side typing, and to prevent leaking fields by accident (passwords, internal flags). The tool for this is *response_model*.

```
from fastapi import FastAPI
from pydantic import BaseModel

class UserIn(BaseModel):
    username: str
    password: str          # we accept it on input
    email: str

class UserOut(BaseModel):
    username: str          # but we never return it
    email: str

app = FastAPI()

@app.post("/users", response_model=UserOut)
def create_user(user: UserIn):
    # ... persist user ...
    return user            # password is filtered out automatically
```

Tip. *response_model* takes precedence over the function's return type for serialisation. Even if you return extra fields, only those declared on the response model leave the server.

Status codes

Set the default success status code with *status_code* on the decorator. Use the *status* module for readable constants.

```
from fastapi import FastAPI, status

app = FastAPI()
```

```
@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item(name: str):
    return {"name": name}

@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int):
    return None    # 204 must have an empty body
```

Returning a non-default status code

To return a different status code for a specific call, inject a *Response* object and set its *status_code* attribute.

```
from fastapi import FastAPI, Response, status

app = FastAPI()

@app.put("/items/{item_id}")
def upsert(item_id: int, name: str, response: Response):
    existed = item_id in {1, 2, 3}
    response.status_code = status.HTTP_200_OK if existed else status.HTTP_201_CREATED
    return {"id": item_id, "name": name}
```

1.8 Forms & File Uploads

JSON is the default, but classic HTML forms and multipart file uploads are common enough that FastAPI gives them dedicated helpers.

Form fields

Use *Form()* for *application/x-www-form-urlencoded* or multipart form fields. The *python-multipart* package (already bundled with the standard install) does the parsing.

```
from typing import Annotated
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/login")
def login(
    username: Annotated[str, Form()],
    password: Annotated[str, Form()],
):
    return {"user": username}
```

File uploads

UploadFile streams the file off the wire — useful for large uploads since the entire file does not need to fit in memory. For small files, *bytes* is fine.

```
from typing import Annotated
from fastapi import FastAPI, File, UploadFile

app = FastAPI()

@app.post("/upload")
async def upload(file: Annotated[UploadFile, File()]):
    contents = await file.read()
    return {
        "filename": file.filename,
        "content_type": file.content_type,
```

```

        "size": len(contents),
    }

@app.post("/upload-many")
async def upload_many(files: Annotated[list[UploadFile], File()]):
    return [{"name": f.filename} for f in files]

```

Note. You cannot accept a JSON body and a file in the same path operation — *multipart/form-data* and *application/json* are mutually exclusive request content types. The usual workaround is to send JSON as a string form field alongside the file and parse it yourself.

1.9 Automatic Docs — Swagger UI & ReDoc

Every FastAPI application exposes two documentation UIs by default:

- **Swagger UI** at */docs* — interactive, lets you try requests from the browser.
- **ReDoc** at */redoc* — reference-style, beautiful for sharing with consumers.
- The raw **OpenAPI 3.1** schema at */openapi.json* — the source for both UIs.

Improving the docs with metadata

```

from fastapi import FastAPI

app = FastAPI(
    title="Bookshelf API",
    summary="Track the books you have read and the ones you mean to.",
    description="A learning project for the FastAPI guide.",
    version="1.0.0",
    contact={"name": "You", "email": "you@example.com"},
    license_info={"name": "MIT"},
)

@app.get("/books", tags=["books"], summary="List all books",
        description="Returns every book, ordered by title.")
def list_books():
    return []

```

tags group endpoints in the UI; *summary* and *description* become the heading and longer text in the Swagger entry. You can use full Markdown in descriptions, including code blocks.

Disabling docs in production (when needed)

If you must hide the docs from public view, pass *docs_url=None* and *redoc_url=None* to the FastAPI constructor. A better pattern is to keep them on but mount the app behind an authenticated reverse proxy.

■ HANDS-ON PROJECT — Personal Bookshelf API

Apply every Phase 1 concept in one small, end-to-end service. Books live in an in-memory dictionary — no database yet.

Goals

- CRUD endpoints for books (create, read, update, delete).
- Path, query, and body parameters all in use.

- Pydantic input and output models, with a hidden internal field.
- Explicit status codes and a custom 404 when a book is missing.
- File upload endpoint to attach a cover image.
- Polished OpenAPI metadata.

Project structure

```
bookshelf/  
  main.py  
  .env/
```

main.py

```

from typing import Annotated
from uuid import UUID, uuid4
from fastapi import FastAPI, HTTPException, Path, Query, status, File, UploadFile
from pydantic import BaseModel, Field

app = FastAPI(
    title="Bookshelf API",
    version="1.0.0",
    summary="Track the books you have read and the ones you mean to.",
)

# ----- Models -----
class BookIn(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    author: str = Field(min_length=1, max_length=120)
    year: int = Field(ge=1450, le=2100)
    pages: int = Field(gt=0, le=10_000)
    read: bool = False

class BookOut(BookIn):
    id: UUID

# ----- In-memory store -----
_books: dict[UUID, BookOut] = {}

# ----- Endpoints -----
@app.get("/books", response_model=list[BookOut], tags=["books"])
def list_books(
    read: Annotated[bool | None, Query(description="Filter by read status")] = None,
    skip: Annotated[int, Query(ge=0)] = 0,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
):
    items = list(_books.values())
    if read is not None:
        items = [b for b in items if b.read == read]
    return items[skip : skip + limit]

@app.post("/books", response_model=BookOut, status_code=status.HTTP_201_CREATED,
          tags=["books"])
def create_book(book: BookIn):
    new = BookOut(id=uuid4(), **book.model_dump())
    _books[new.id] = new
    return new

@app.get("/books/{book_id}", response_model=BookOut, tags=["books"])
def get_book(book_id: Annotated[UUID, Path()]):
    if book_id not in _books:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "book not found")
    return _books[book_id]

@app.put("/books/{book_id}", response_model=BookOut, tags=["books"])
def replace_book(book_id: UUID, book: BookIn):
    if book_id not in _books:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "book not found")
    updated = BookOut(id=book_id, **book.model_dump())
    _books[book_id] = updated
    return updated

@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT,
            tags=["books"])
def delete_book(book_id: UUID):

```



```
    if book_id not in _books:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "book not found")
    del _books[book_id]

@app.post("/books/{book_id}/cover", tags=["books"])
async def upload_cover(book_id: UUID, file: UploadFile = File(...)):
    if book_id not in _books:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "book not found")
    data = await file.read()
    return {"book_id": book_id, "filename": file.filename, "size": len(data)}
```

Run it

```
fastapi dev main.py
```

Open `/docs`. Create a book, list, fetch by id, update, upload a cover, delete. Try invalid inputs and watch the 422 errors. This single file exercises everything Phase 1 covered. You are ready for the harder material.

PHASE 2

Intermediate

Dependency injection, project structure, async I/O, databases, and authentication — the building blocks of a real service.

2.1 Dependency Injection (Depends)

Dependency injection (DI) is FastAPI's mechanism for decoupling a path operation from the things it needs. A dependency is just a function — sync or async — that FastAPI runs before the path operation, passing its return value to your code. Common dependencies: getting the current user, opening a database session, parsing common query parameters, enforcing permissions.

```
from typing import Annotated
from fastapi import FastAPI, Depends

app = FastAPI()

def pagination(skip: int = 0, limit: int = 10) -> dict:
    return {"skip": skip, "limit": limit}

@app.get("/items")
def list_items(page: Annotated[dict, Depends(pagination)]):
    return page

@app.get("/users")
def list_users(page: Annotated[dict, Depends(pagination)]):
    return page
```

Both endpoints now accept *skip* and *limit* as query parameters — FastAPI extracts them, calls *pagination()*, and passes you the result. The Swagger UI reflects the parameters automatically.

Class-based dependencies

Any callable works. A class with an `__init__` is often the cleanest way to group related parameters.

```
class Pagination:
    def __init__(self, skip: int = 0, limit: int = 10):
        self.skip = skip
        self.limit = limit

@app.get("/items")
def list_items(p: Annotated[Pagination, Depends()]):
    return {"skip": p.skip, "limit": p.limit}
```

Note. When the dependency class is also the type annotation, you can write `Depends()` without passing the callable — FastAPI uses the type.

Sub-dependencies

Dependencies can depend on other dependencies. FastAPI assembles the whole graph and caches each result for the duration of a single request, so a dependency called from three places only runs once.

```
def get_db():
    db = SessionLocal()
    try:
        yield db          # value passed to the path operation
    finally:
        db.close()        # cleanup runs after the response is sent

def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
    db: Annotated[Session, Depends(get_db)],
) -> User:
```

```
...

@app.get("/me")
def me(user: Annotated[User, Depends(get_current_user)]):
    return user
```

Dependencies that *yield* are **resource-managing** dependencies: code before the yield runs as setup, the yielded value goes to the caller, and code after the yield runs as teardown. This is how you safely open and close database sessions, files, or remote connections.

Application-wide dependencies

Attach dependencies to the FastAPI instance or to a router for cross-cutting concerns:

```
from fastapi import FastAPI, Depends, Header, HTTPException

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != "secret":
        raise HTTPException(401, "invalid api key")

app = FastAPI(dependencies=[Depends(verify_api_key)])
```

2.2 Routers & Project Structure (APIRouter)

A single *main.py* works for a tutorial. For anything real you want to split the code into modules grouped by domain — users, items, orders. *APIRouter* is to a FastAPI app what a *Blueprint* is to Flask: a collection of routes that can be mounted at a prefix and shared across files.

A practical layout

```
myapp/
  app/
    __init__.py
    main.py           # FastAPI() instance, mounts routers
    settings.py       # Pydantic Settings
    deps.py           # shared dependencies (db, auth)
    db.py             # engine, SessionLocal
    models.py         # SQLAlchemy models
    schemas.py        # Pydantic models
    routers/
      __init__.py
      users.py
      items.py
    tests/
  pyproject.toml
```

Defining a router

```
# app/routers/users.py
from fastapi import APIRouter, Depends
from .. import schemas, deps

router = APIRouter(prefix="/users", tags=["users"])

@router.get("", response_model=list[schemas.User])
def list_users():
    return []

@router.get("/{user_id}", response_model=schemas.User)
def get_user(user_id: int):
    ...
```

```
# app/main.py
from fastapi import FastAPI
from .routers import users, items

app = FastAPI(title="My App")
app.include_router(users.router)
app.include_router(items.router)
```

Tip. Always give routers a prefix and at least one tag. Prefixes keep URLs tidy; tags keep the Swagger UI navigable.

2.3 Async / Await in FastAPI

FastAPI is ASGI-native. You can declare a path operation as either a regular function or an *async def* function, and FastAPI handles each correctly:

- **async def** — runs on the event loop. Use this when you *await* something: an async database driver, *httpx.AsyncClient*, *aiofiles*, Redis async client, etc.
- **def** — runs in a thread pool, so it does not block the event loop. Use this when your code is synchronous and may block (e.g. classic SQLAlchemy, *requests*, blocking SDKs).

The wrong combination is the one to avoid: a blocking call inside an *async def*.

```
import asyncio
import time
import httpx
from fastapi import FastAPI

app = FastAPI()

# OK: async I/O inside async def
@app.get("/weather")
async def weather():
    async with httpx.AsyncClient() as client:
        r = await client.get("https://api.example.com/weather")
        return r.json()

# OK: blocking work inside def (FastAPI runs it in a thread)
@app.get("/report")
def report():
    time.sleep(2)          # would block the loop if inside async def
    return {"done": True}

# WRONG: blocking call in an async function – freezes the event loop
@app.get("/bad")
async def bad():
    time.sleep(2)          # NEVER do this in async def
    return {"done": True}
```

Note. If you must run a blocking function from an async path operation, wrap it: *await asyncio.to_thread(blocking_fn, arg1, arg2)*. This is the canonical escape hatch.

2.4 Databases with SQLAlchemy (sync)

SQLAlchemy 2.x is the standard Python ORM. The synchronous flavour is mature, well-documented, and pairs naturally with FastAPI's thread-pool execution of regular *def* functions.

```
pip install sqlalchemy psycopg2-binary
```

Engine and session

```
# app/db.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

DATABASE_URL = "postgresql+psycopg2://user:pass@localhost/mydb"
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):
    pass
```

Model

```
# app/models.py
from sqlalchemy.orm import Mapped, mapped_column
from .db import Base

class Item(Base):
    __tablename__ = "items"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    price: Mapped[float]
```

Session dependency

```
# app/deps.py
from typing import Annotated
from fastapi import Depends
from sqlalchemy.orm import Session
from .db import SessionLocal

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

DB = Annotated[Session, Depends(get_db)]
```

Endpoint using the session

```
from fastapi import APIRouter, HTTPException
from sqlalchemy import select
from .deps import DB
from . import models, schemas

router = APIRouter(prefix="/items", tags=["items"])

@router.post("", response_model=schemas.Item, status_code=201)
def create(payload: schemas.ItemIn, db: DB):
    item = models.Item(**payload.model_dump())
    db.add(item)
    db.commit()
    db.refresh(item)
    return item

@router.get("/{item_id}", response_model=schemas.Item)
def read(item_id: int, db: DB):
    item = db.get(models.Item, item_id)
    if item is None:
        raise HTTPException(404, "not found")
    return item
```

Tip. Pair SQLAlchemy ORM models with Pydantic schemas configured with `model_config = ConfigDict(from_attributes=True)`. That lets FastAPI serialise an ORM instance through the response model without manual conversion.

2.5 Async Databases

When throughput matters and your database driver supports it, switch to the async flavour. SQLAlchemy 2.x ships first-class async support; the driver of choice for PostgreSQL is *asyncpg*.

```
pip install sqlalchemy[asyncio] asyncpg

# app/db.py (async variant)
from sqlalchemy.ext.asyncio import (
    AsyncSession, async_sessionmaker, create_async_engine,
)
from sqlalchemy.orm import DeclarativeBase

DATABASE_URL = "postgresql+asyncpg://user:pass@localhost/mydb"
engine = create_async_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

# app/deps.py
from typing import Annotated
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession
from .db import SessionLocal

async def get_db():
    async with SessionLocal() as session:
        yield session

DB = Annotated[AsyncSession, Depends(get_db)]

# Async endpoint
from sqlalchemy import select
```

```

from fastapi import APIRouter
from .deps import DB
from . import models, schemas

router = APIRouter()

@router.get("/items", response_model=list[schemas.Item])
async def list_items(db: DB):
    result = await db.execute(select(models.Item))
    return result.scalars().all()

```

Note. Async sessions cannot be used implicitly with lazy loading of relationships. Use `selectinload()` or `joinedload()` in the query, or wrap relationship access in `await session.refresh(obj, ["rel"])`.

2.6 Alembic Migrations

ORM models describe what the schema *should* be; migrations evolve the live database to match. Alembic is the migration tool from the SQLAlchemy author and the standard choice.

```

pip install alembic
alembic init alembic

```

Edit `alembic.ini` to set `sqlalchemy.url`, or — better — let the env file read it from your settings module:

```

# alembic/env.py (key lines)
from app.db import Base
from app.models import * # so Alembic sees them
from app.settings import settings

config.set_main_option("sqlalchemy.url", settings.database_url_sync)
target_metadata = Base.metadata

```

Common commands

```

# Generate a migration from current vs. model diff
alembic revision --autogenerate -m "add items table"

# Apply migrations to head
alembic upgrade head

# Roll back one step
alembic downgrade -1

# Show current revision
alembic current

```

Tip. Always review autogenerated migrations before committing. Alembic catches most schema changes but is conservative with renames (it sees a drop+add) and indexes.

2.7 Authentication — OAuth2 & JWT

FastAPI ships with security helpers that integrate with the OpenAPI security scheme so the Swagger UI gets an *Authorize* button for free. The most common production pattern is OAuth2 with the password flow plus JWT bearer tokens.

```

pip install "passlib[bcrypt]" "python-jose[cryptography]"
# app/auth.py
from datetime import datetime, timedelta, timezone

```



```

from typing import Annotated
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from passlib.context import CryptContext

SECRET_KEY = "replace-me-with-env-secret"
ALGORITHM = "HS256"
TOKEN_TTL = timedelta(minutes=30)

pwd_ctx = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")

def hash_password(p: str) -> str:
    return pwd_ctx.hash(p)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_ctx.verify(plain, hashed)

def create_access_token(sub: str) -> str:
    payload = {"sub": sub, "exp": datetime.now(timezone.utc) + TOKEN_TTL}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def get_current_username(token: Annotated[str, Depends(oauth2_scheme)]) -> str:
    err = HTTPException(status.HTTP_401_UNAUTHORIZED, "invalid token",
                        headers={"WWW-Authenticate": "Bearer"})
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str | None = payload.get("sub")
        if username is None:
            raise err
        return username
    except JWTError:
        raise err

# app/routers/auth.py
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from .. import auth

router = APIRouter(prefix="/auth", tags=["auth"])

# Toy user store; replace with a database lookup.
_users = {"alice": auth.hash_password("wonderland")}

@router.post("/login")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]):
    pwd_hash = _users.get(form.username)
    if not pwd_hash or not auth.verify_password(form.password, pwd_hash):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED,
                            "bad username or password")
    return {"access_token": auth.create_access_token(form.username),
            "token_type": "bearer"}

```

Protect any endpoint by depending on `get_current_username`. The Swagger UI now shows an Authorize button; logging in there populates every subsequent request.

```

@app.get("/me")
def me(user: Annotated[str, Depends(auth.get_current_username)]):
    return {"username": user}

```

Tip. Never check passwords in plain text and never store the secret key in source. Load it from an environment variable. We will cover Pydantic Settings and secret management in Phase 4.

2.8 Middleware — CORS & Custom

Middleware sits between the HTTP server and your route, seeing every request and response. Use it for cross-cutting concerns: CORS, request IDs, timing, gzip, authentication checks that apply globally.

CORS

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Note. Setting `allow_origins=["*"]` with `allow_credentials=True` is rejected by browsers. List explicit origins in production.

Custom middleware

```
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time-ms"] = f"{{(time.perf_counter() - start) * 1000:.2f}}"
    return response
```

2.9 Error Handling & Custom Exception Handlers

`HTTPException` is the workhorse: raise it from anywhere with a status code and a detail message. For domain exceptions you want to convert to specific HTTP responses, register an exception handler.

```
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse

app = FastAPI()

class InsufficientFunds(Exception):
    def __init__(self, account_id: int, requested: float):
        self.account_id = account_id
        self.requested = requested
```

```

@app.exception_handler(InsufficientFunds)
async def funds_handler(request: Request, exc: InsufficientFunds):
    return JSONResponse(
        status_code=status.HTTP_409_CONFLICT,
        content={
            "error": "insufficient_funds",
            "account_id": exc.account_id,
            "requested": exc.requested,
        },
    )

@app.post("/withdraw")
def withdraw(account_id: int, amount: float):
    if amount > 100:
        raise InsufficientFunds(account_id, amount)
    return {"ok": True}

```

Override the default validation error format

FastAPI's default 422 response is detailed but verbose. Customise it by overriding the handler for *RequestValidationError*.

```

from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={"error": "validation_failed", "details": exc.errors()},
    )

```

2.10 Background Tasks

For short post-response work — sending an email, writing an audit log entry, updating a counter — FastAPI's *BackgroundTasks* is enough. The task runs after the response is sent, on the same worker, so do not use it for anything you cannot afford to lose if the process dies. For durable work, use Celery (Phase 3).

```

from fastapi import FastAPI, BackgroundTasks

app = FastAPI()

def write_log(message: str):
    with open("audit.log", "a") as f:
        f.write(message + "\n")

@app.post("/orders")
def create_order(payload: dict, bg: BackgroundTasks):
    # ... persist order ...
    bg.add_task(write_log, f"order created: {payload}")
    return {"status": "accepted"}

```

■ HANDS-ON PROJECT — Task Manager API

A multi-user to-do service with JWT auth, SQLAlchemy persistence, Alembic migrations, custom errors, and email-on-create background tasks.

Goals

- Users register and log in; each user owns their own tasks.
- Tasks persisted in PostgreSQL via SQLAlchemy.
- Schema managed by Alembic.
- OAuth2 password flow with JWT bearer tokens.
- Modular routers under *app/routers/*.
- Background task that simulates sending a welcome email on signup.
- Custom 404 and 409 handlers for domain errors.

Project layout

```
tasks_api/
  app/
    __init__.py
    main.py
    db.py
    models.py
    schemas.py
    auth.py
    deps.py
    routers/
      __init__.py
      users.py
      tasks.py
  alembic/
  alembic.ini
```

Models (app/models.py)

```
from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .db import Base

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True, index=True)
    password_hash: Mapped[str]
    tasks: Mapped[list["Task"]] = relationship(back_populates="owner")

class Task(Base):
    __tablename__ = "tasks"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    done: Mapped[bool] = mapped_column(default=False)
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    owner: Mapped[User] = relationship(back_populates="tasks")
```

Schemas (app/schemas.py)

```
from pydantic import BaseModel, ConfigDict, Field

class UserCreate(BaseModel):
    username: str = Field(min_length=3, max_length=50)
    password: str = Field(min_length=8)

class UserOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    username: str

class TaskCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)

class TaskOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    title: str
    done: bool
```

Users router (app/routers/users.py)

```
from typing import Annotated
from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy import select
from .. import auth, models, schemas
from ..deps import DB

router = APIRouter(prefix="/users", tags=["users"])

def send_welcome_email(username: str):
    # In a real app you would call an email provider here.
    print(f"[email] welcome, {username}!")

@router.post("/register", response_model=schemas.UserOut, status_code=201)
def register(payload: schemas.UserCreate, bg: BackgroundTasks, db: DB):
    if db.scalar(select(models.User).where(models.User.username == payload.username)):
        raise HTTPException(status.HTTP_409_CONFLICT, "username taken")
    user = models.User(username=payload.username,
                       password_hash=auth.hash_password(payload.password))
    db.add(user); db.commit(); db.refresh(user)
    bg.add_task(send_welcome_email, user.username)
    return user

@router.post("/login")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()], db: DB):
    user = db.scalar(select(models.User).where(models.User.username == form.username))
    if not user or not auth.verify_password(form.password, user.password_hash):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "bad credentials")
    return {"access_token": auth.create_access_token(user.username),
            "token_type": "bearer"}
```

Tasks router (app/routers/tasks.py)

```

from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy import select
from .. import auth, models, schemas
from ..deps import DB

router = APIRouter(prefix="/tasks", tags=["tasks"])

def current_user(
    username: Annotated[str, Depends(auth.get_current_username)],
    db: DB,
) -> models.User:
    user = db.scalar(select(models.User).where(models.User.username == username))
    if user is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "unknown user")
    return user

User = Annotated[models.User, Depends(current_user)]

@router.get("", response_model=list[schemas.TaskOut])
def list_my_tasks(user: User, db: DB):
    return db.scalars(select(models.Task).where(models.Task.owner_id == user.id)).all()

@router.post("", response_model=schemas.TaskOut, status_code=201)
def create_task(payload: schemas.TaskCreate, user: User, db: DB):
    t = models.Task(title=payload.title, owner_id=user.id)
    db.add(t); db.commit(); db.refresh(t)
    return t

@router.patch("/{task_id}/done", response_model=schemas.TaskOut)
def mark_done(task_id: int, user: User, db: DB):
    t = db.get(models.Task, task_id)
    if t is None or t.owner_id != user.id:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "task not found")
    t.done = True; db.commit(); db.refresh(t)
    return t

```

Wire it up (app/main.py)

```

from fastapi import FastAPI
from .routers import users, tasks

app = FastAPI(title="Task Manager API", version="1.0.0")
app.include_router(users.router)
app.include_router(tasks.router)

```

Migration and run

```

alembic revision --autogenerate -m "initial schema"
alembic upgrade head
fastapi dev app/main.py

```

Register a user, log in to get a token, click *Authorize* in */docs*, and create tasks. You now have the skeleton of essentially any CRUD service that needs accounts.

PHASE 3

Advanced

*Real-time channels, lifespan, caching, queues, structured logs, testing, and observability
— the craft of a serious service.*

3.1 Advanced Pydantic v2 — Validators & Computed Fields

Pydantic v2 introduces a far more powerful (and faster) validation pipeline than v1. Two features deserve fluency: *field_validator/model_validator* for custom checks, and *computed_field* for derived values that should appear in the serialised output.

Field validators

```
from pydantic import BaseModel, field_validator

class SignUp(BaseModel):
    username: str
    password: str
    password_confirm: str

    @field_validator("username")
    @classmethod
    def lowercase_username(cls, v: str) -> str:
        if not v.isalnum():
            raise ValueError("username must be alphanumeric")
        return v.lower()
```

Model validators (cross-field)

Use *model_validator(mode="after")* when the rule involves multiple fields:

```
from pydantic import BaseModel, model_validator

class SignUp(BaseModel):
    password: str
    password_confirm: str

    @model_validator(mode="after")
    def passwords_match(self) -> "SignUp":
        if self.password != self.password_confirm:
            raise ValueError("passwords do not match")
        return self
```


Computed fields

```
from datetime import date
from pydantic import BaseModel, computed_field

class Person(BaseModel):
    first_name: str
    last_name: str
    birth_date: date

    @computed_field
    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"

    @computed_field
    @property
    def age(self) -> int:
        today = date.today()
        years = today.year - self.birth_date.year
        if (today.month, today.day) < (self.birth_date.month, self.birth_date.day):
            years -= 1
        return years
```

`full_name` and `age` appear in JSON output and in the OpenAPI schema, but you never store them — they are derived on the fly.

Pydantic Settings for configuration

```
# app/settings.py
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="APP_")
    database_url: str
    secret_key: str = Field(min_length=32)
    debug: bool = False

settings = Settings()
```

Note. Install with `pip install pydantic-settings`. Environment variables prefixed `APP_` (e.g. `APP_DATABASE_URL`) auto-populate the fields. Missing required fields fail fast at startup, which is exactly what you want.

3.2 WebSockets

WebSockets provide a bidirectional, long-lived channel between client and server. FastAPI exposes them with a clean decorator and a `WebSocket` object you *await* on.

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

@app.websocket("/ws/echo")
async def echo(ws: WebSocket):
    await ws.accept()
    try:
        while True:
            msg = await ws.receive_text()
```

```

        await ws.send_text(f"echo: {msg}")
    except WebSocketDisconnect:
        return

```

A broadcast room

Real applications need to fan messages out to many connected clients. The pattern is a connection manager that tracks active sockets.

```

from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

class Hub:
    def __init__(self):
        self.connections: list[WebSocket] = []

    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.connections.append(ws)

    def disconnect(self, ws: WebSocket):
        self.connections.remove(ws)

    async def broadcast(self, message: str):
        for ws in list(self.connections):
            try:
                await ws.send_text(message)
            except RuntimeError:
                self.disconnect(ws)

hub = Hub()

@app.websocket("/ws/chat/{room}")
async def chat(ws: WebSocket, room: str):
    await hub.connect(ws)
    try:
        while True:
            msg = await ws.receive_text()
            await hub.broadcast(f"[{room}] {msg}")
    except WebSocketDisconnect:
        hub.disconnect(ws)

```

Note. An in-process hub is fine for a single worker. To broadcast across multiple workers or hosts, back the hub with Redis pub/sub or NATS.

3.3 Lifespan & Startup/Shutdown Events

When the app starts you often need to open resources — a database connection pool, a Redis client, a metrics registry. When it stops you need to close them. Modern FastAPI uses an *asynccontextmanager* called *lifespan*.

```

from contextlib import asynccontextmanager
from fastapi import FastAPI
import httpx

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ---- startup ----
    app.state.http = httpx.AsyncClient(timeout=10.0)

```

```

    print("started")
    yield
    # ---- shutdown ----
    await app.state.http.aclose()
    print("stopped")

app = FastAPI(lifespan=lifespan)

@app.get("/ping")
async def ping():
    r = await app.state.http.get("https://example.com")
    return {"status": r.status_code}

```

Tip. Pass resources to handlers via `app.state` or — better — via dependencies that return them. Avoid module-level globals; they make testing harder.

3.4 Rate Limiting with `slowapi`

Protect endpoints from abuse with *slowapi*, a FastAPI/Starlette-compatible rate limiter built on the popular *limits* library. It supports per-IP, per-user, per-endpoint limits, with Redis as a shared backend in multi-worker deployments.

```

pip install slowapi

from fastapi import FastAPI, Request
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address,
                  storage_uri="redis://localhost:6379") # or memory://

app = FastAPI()
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)

@app.get("/search")
@limiter.limit("10/minute")
async def search(request: Request, q: str = ""):
    return {"q": q}

```

Note. The request parameter on rate-limited endpoints is required by *slowapi* — even if you do not otherwise use it.

3.5 Caching & Redis

Most performance wins in API services come from not doing work twice. Redis is the default cache and the standard async client is *redis-py* (which absorbed *aioredis* from version 4.2 onward).

```

pip install redis

import json
from contextlib import asynccontextmanager
from fastapi import FastAPI, Depends
import redis.asyncio as redis

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = redis.from_url("redis://localhost", decode_responses=True)
    yield

```

```

    await app.state.redis.aclose()

app = FastAPI(lifespan=lifespan)

async def cache():
    return app.state.redis

@app.get("/expensive/{key}")
async def expensive(key: str, r: redis.Redis = Depends(cache)):
    cached = await r.get(f"exp:{key}")
    if cached:
        return json.loads(cached)
    result = {"key": key, "value": compute_slowly(key)}
    await r.set(f"exp:{key}", json.dumps(result), ex=300)  # TTL 5 minutes
    return result

```

Tip. For decorator-style caching on endpoint or function results, the `fastapi-cache2` package wraps this pattern and supports key builders, TTLs, and namespaces.

3.6 Task Queues with Celery

`BackgroundTasks` handles fire-and-forget work that fits inside the request process. For anything you need to survive a restart, retry on failure, schedule, or fan out across worker machines — use Celery.

```

pip install "celery[redis]"

# app/worker.py
from celery import Celery

celery_app = Celery(
    "tasks",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)
celery_app.conf.task_serializer = "json"
celery_app.conf.task_track_started = True

@celery_app.task(bind=True, max_retries=3, default_retry_delay=10)
def send_email(self, to: str, subject: str, body: str) -> str:
    try:
        # actual SMTP call here
        return f"sent to {to}"
    except Exception as exc:
        raise self.retry(exc=exc)

# app/routers/orders.py
from fastapi import APIRouter
from ..worker import send_email

router = APIRouter()

@router.post("/orders")
def place_order(payload: dict):
    # ... persist order ...
    task = send_email.delay(payload["email"], "Order received",
                           "Thanks for your purchase.")
    return {"task_id": task.id}

# Run a worker in another terminal
celery -A app.worker.celery_app worker --loglevel=info

# Periodic / scheduled tasks: run beat alongside

```

```
celery -A app.worker.celery_app beat --loglevel=info
```

3.7 Logging & Structured Logs

Plain text logs are fine in development. In production you want structured logs — JSON, one event per line — so observability tools can index and filter them. The *structlog* library is the most ergonomic option in modern Python.

```
pip install structlog

# app/logging_config.py
import logging
import structlog

def configure_logging(debug: bool = False) -> None:
    logging.basicConfig(format="%(message)s", level=logging.DEBUG if debug else logging.INFO)
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.StackInfoRenderer(),
            structlog.processors.format_exc_info,
            structlog.processors.JSONRenderer(),
        ],
        wrapper_class=structlog.make_filtering_bound_logger(
            logging.DEBUG if debug else logging.INFO),
        cache_logger_on_first_use=True,
    )

log = structlog.get_logger()

# Middleware adding a request ID to every log line
import uuid
import structlog
from fastapi import FastAPI, Request

app = FastAPI()
log = structlog.get_logger()

@app.middleware("http")
async def request_context(request: Request, call_next):
    request_id = request.headers.get("x-request-id") or uuid.uuid4().hex
    structlog.contextvars.bind_contextvars(
        request_id=request_id, path=request.url.path, method=request.method)
    try:
        response = await call_next(request)
        log.info("request_complete", status=response.status_code)
        response.headers["x-request-id"] = request_id
        return response
    finally:
        structlog.contextvars.clear_contextvars()
```

3.8 Testing — pytest, TestClient, Mocking

TestClient wraps your application in an in-process HTTP client (built on *httpx* internally) so tests run without a network or a server. Combined with *pytest* and FastAPI's *dependency_overrides*, you get fast, isolated, expressive tests.

```
pip install pytest httpx
```

```
# tests/test_books.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_and_read():
    r = client.post("/books", json={
        "title": "Dune", "author": "Herbert",
        "year": 1965, "pages": 412})
    assert r.status_code == 201
    book = r.json()
    r2 = client.get(f"/books/{book['id']}")
    assert r2.status_code == 200
    assert r2.json()["title"] == "Dune"

def test_validation_error():
    r = client.post("/books", json={"title": "", "year": 1965, "pages": 1})
    assert r.status_code == 422
```

Overriding dependencies for tests

```
from app.main import app
from app.deps import get_db

def override_get_db():
    # yield a sqlite test session, an in-memory fake, etc.
    ...

app.dependency_overrides[get_db] = override_get_db
```

Async tests

For async clients use *httpx.AsyncClient* with *pytest-anyio*:

```
import pytest
import httpx
from app.main import app

@pytest.mark.anyio
async def test_async():
    transport = httpx.ASGITransport(app=app)
    async with httpx.AsyncClient(transport=transport, base_url="http://test") as ac:
        r = await ac.get("/books")
        assert r.status_code == 200
```

3.9 Monitoring — Prometheus & Sentry

Two complementary tools cover most observability needs: Prometheus scrapes numeric metrics from your service (request rates, latencies, queue depths); Sentry captures exceptions with full stack traces and request context.

Prometheus

```
pip install prometheus-fastapi-instrumentator

from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI()
Instrumentator().instrument(app).expose(app, endpoint="/metrics")
```

Out of the box you get request count, latency histogram, and in-flight gauges, all labelled by path and method, scrapable from `/metrics`.

Sentry

```
pip install "sentry-sdk[fastapi]"
import sentry_sdk
from sentry_sdk.integrations.fastapi import FastApiIntegration
from sentry_sdk.integrations.starlette import StarletteIntegration

sentry_sdk.init(
    dsn="https://&key&@o0.ingest.sentry.io/0",
    traces_sample_rate=0.1,
    integrations=[StarletteIntegration(), FastApiIntegration()],
    environment="production",
)
```

■ HANDS-ON PROJECT — Real-time Chat & Notifications

Combine WebSockets, Redis pub/sub, Celery, structured logs, rate limiting, and a Prometheus endpoint into one cohesive service.

Goals

- Users connect to a WebSocket room and exchange messages in real time.
- Messages broadcast across multiple workers via Redis pub/sub.
- Sending a message also enqueues a Celery task to persist it.
- Each connection is rate-limited; abusive clients get disconnected.
- Lifespan opens Redis and HTTP clients; closes them cleanly.
- Structured logs with a per-request ID.
- Prometheus metrics at `/metrics`.

Core lifespan and Redis pub/sub broadcaster

```

from contextlib import asynccontextmanager
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
import asyncio, json
import redis.asyncio as redis
from prometheus_fastapi_instrumentator import Instrumentator
from .worker import persist_message
from .logging_config import configure_logging, log

configure_logging()

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = redis.from_url("redis://localhost", decode_responses=True)
    app.state.connections: dict[str, set[WebSocket]] = {}
    pubsub = app.state.redis.pubsub()
    await pubsub.subscribe("chat")
    app.state.reader_task = asyncio.create_task(_reader(app, pubsub))
    yield
    app.state.reader_task.cancel()
    await pubsub.unsubscribe("chat")
    await app.state.redis.aclose()

async def _reader(app, pubsub):
    async for msg in pubsub.listen():
        if msg.get("type") != "message":
            continue
        data = json.loads(msg["data"])
        room = data["room"]
        for ws in list(app.state.connections.get(room, [])):
            try:
                await ws.send_json(data)
            except Exception:
                pass

app = FastAPI(lifespan=lifespan, title="Real-time Chat")
Instrumentator().instrument(app).expose(app, endpoint="/metrics")

@app.websocket("/ws/{room}/{user}")
async def chat(ws: WebSocket, room: str, user: str):
    await ws.accept()
    app.state.connections.setdefault(room, set()).add(ws)
    log.info("ws_connected", room=room, user=user)
    try:
        while True:
            text = await ws.receive_text()
            event = {"room": room, "user": user, "text": text}
            await app.state.redis.publish("chat", json.dumps(event))
            persist_message.delay(room, user, text)
    except WebSocketDisconnect:
        app.state.connections[room].discard(ws)
        log.info("ws_disconnected", room=room, user=user)

```

Run two instances on different ports behind a load balancer to prove the pub/sub fan-out: messages from a client connected to instance A appear at clients connected to instance B.

PHASE 4

Professional

*Docker, security, versioning, OpenAPI for SDKs, performance, microservices, CI/CD —
production engineering.*

4.1 Docker & Deployment (gunicorn + uvicorn)

FastAPI's recommended production server is Uvicorn — an ASGI server built on *uvloop* and *httptools*. For multi-process serving with graceful restarts and worker management, the long-standing pattern is to run Uvicorn *workers* under Gunicorn, the battle-tested process manager.

Dockerfile

```
# syntax=docker/dockerfile:1.7
FROM python:3.13-slim AS base

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

# install build deps for any wheels that need them, then drop them
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev curl ca-certificates && \
    rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
    && pip install --no-cache-dir "gunicorn>=23" "uvicorn[standard]>=0.30"

COPY app ./app
COPY alembic ./alembic
COPY alembic.ini ./

RUN adduser --disabled-password --gecos '' appuser
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -fsS http://localhost:8000/health || exit 1

CMD ["gunicorn", "app.main:app", \
    "-k", "uvicorn.workers.UvicornWorker", \
    "-w", "4", \
    "-b", "0.0.0.0:8000", \
    "--access-logfile", "-", \
    "--timeout", "60"]
```

Choosing the worker count

A common starting heuristic is $(2 \times \text{CPU cores}) + 1$ workers. For an async app with mostly I/O-bound endpoints, fewer workers with more concurrency per worker is often better — start with one worker per core and measure.

docker-compose for local stacks

```
services:
  api:
    build: .
    environment:
      APP_DATABASE_URL: postgresql+psycopg2://app:app@db/app
      APP_SECRET_KEY: ${APP_SECRET_KEY}
    depends_on: [db, redis]
    ports: ["8000:8000"]
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    volumes: ["pgdata:/var/lib/postgresql/data"]
  redis:
    image: redis:7-alpine
volumes:
  pgdata:
```

4.2 Security Best Practices — HTTPS, Secrets, OWASP

Transport

- Terminate TLS at a reverse proxy (Caddy, Nginx, Traefik, a cloud load balancer). FastAPI should not handle certificates directly.
- Force HTTPS with `HTTPSRedirectMiddleware`; enforce HSTS at the proxy.
- Set `Secure`, `HttpOnly`, `SameSite=lax` on any cookies you do issue.

```
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
```

```
app.add_middleware(HTTPSRedirectMiddleware)
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["api.example.com"])
```

Secrets

- Never commit secrets. Use environment variables loaded by Pydantic Settings.
- In production, source secrets from a manager: AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, Doppler.
- Rotate keys (JWT signing keys, database passwords) on a schedule; build the app so rotation is a config change, not a deploy.

OWASP top considerations for APIs

- **Broken object level authorization** — always check the requesting user owns the object, not just that they are logged in.
- **Broken authentication** — use battle-tested libraries (Authlib, fastapi-users) rather than rolling your own.
- **Excessive data exposure** — use response models to whitelist fields.

- **Lack of resource limits** — rate limit, size-limit uploads, set per-request timeouts.
- **Injection** — always use parameterised queries (SQLAlchemy does this by default).
- **Improper assets management** — keep an inventory of versions and deprecate old endpoints.

Tip. Run `pip-audit` in CI to flag dependencies with known CVEs. Run `bandit` to scan your own code for common pitfalls.

4.3 API Versioning & Design

Versioning is the contract you offer your consumers. The two production-friendly approaches are URL path versioning (`/v1/...`, `/v2/...`) and header versioning (`Accept: application/vnd.app.v1+json`). Path versioning is simpler, more cacheable, and easier to test — start there unless you have a strong reason not to.

```
from fastapi import FastAPI
from .routers.v1 import users as users_v1, items as items_v1
from .routers.v2 import users as users_v2

app = FastAPI()
app.include_router(users_v1.router, prefix="/v1")
app.include_router(items_v1.router, prefix="/v1")
app.include_router(users_v2.router, prefix="/v2")
```

Design principles that age well

- Use plural nouns for collections (`/users`, not `/user`).
- Use HTTP verbs for actions; do not put them in the URL (`POST /users`, not `POST /users/create`).
- Filter, sort, paginate via query parameters; never invent path-segment conventions for these.
- Use 4xx for client errors and 5xx for server errors. Never return 200 OK with an error payload.
- Use ISO-8601 datetimes in UTC, identifiers as UUIDs or opaque strings, and money as a (decimal, currency) pair — not floats.
- Document deprecation in the response (a `Deprecation` header and a `Sunset` header are the emerging standards).

4.4 OpenAPI Schema & SDK Generation

FastAPI's OpenAPI 3.1 schema is not just for the docs — it is a complete machine-readable contract. You can generate type-safe SDKs for almost any language directly from it.

Dump the schema at build time

```
# scripts/export_openapi.py
import json
from app.main import app

with open("openapi.json", "w") as fh:
    json.dump(app.openapi(), fh, indent=2)
```

Generate a TypeScript client

```
# Recommended in 2026: hey-api/openapi-ts
npm install -D @hey-api/openapi-ts
npx openapi-ts -i openapi.json -o src/api -c @hey-api/client-fetch
```

The output is a fully-typed TypeScript module: request builders, response types, and parameter types all match your Pydantic models. Frontend developers get compile-time safety against backend changes.

Generate a Python client

```
pip install openapi-python-client
openapi-python-client generate --path openapi.json
```

Customising operation IDs

Auto-generated operation IDs look like *list_items_items_get*. Override them with a generator for clean SDK method names.

```
from fastapi.routing import APIRoute

def custom_operation_id(route: APIRoute) -> str:
    return f"{route.tags[0]}_{route.name}" if route.tags else route.name

for r in app.routes:
    if isinstance(r, APIRoute):
        r.operation_id = custom_operation_id(r)
```

4.5 Performance Tuning

The order of attack for a slow FastAPI service is almost always: (1) measure, (2) the database, (3) blocking calls in async paths, (4) serialisation, (5) framework overhead. Start at the top — do not guess.

Profiling

```
pip install pyinstrument

from fastapi import FastAPI, Request
from pyinstrument import Profiler

app = FastAPI()

@app.middleware("http")
async def profile_request(request: Request, call_next):
    profile = request.query_params.get("profile") == "1"
    if not profile:
        return await call_next(request)
    with Profiler(interval=0.001, async_mode="enabled") as p:
        response = await call_next(request)
        print(p.output_text(unicode=True, color=False))
    return response
```

Connection pooling

- Database: set *pool_size* and *max_overflow* on the engine; never default the pool in production.
- HTTP outbound: reuse a single *httpx.AsyncClient* for the whole app, opened in lifespan.
- Redis: *redis.asyncio.from_url* already pools by default — do not create a client per request.

Serialisation

- Pydantic v2 is already fast. The bottleneck is usually the size of responses — paginate aggressively.
- For very hot endpoints returning fixed shapes, you can swap to `ORJSONResponse` as the default response class — typically 20–40% faster JSON encoding.

```
from fastapi import FastAPI
from fastapi.responses import ORJSONResponse

app = FastAPI(default_response_class=ORJSONResponse)
```

Benchmark, then deploy

```
# wrk is the standard one-shot HTTP benchmark
wrk -t4 -c100 -d30s http://localhost:8000/items
```

4.6 Microservices with FastAPI

FastAPI is a natural fit for microservices because services are small, the framework's footprint is tiny, and OpenAPI gives you a contract per service for free. The interesting questions in microservices are about *between* the services, not the framework.

Communication patterns

- **Synchronous** — HTTP/JSON or gRPC between services. Easy to reason about; fragile when a downstream is slow.
- **Asynchronous** — events on a message bus (Kafka, NATS, RabbitMQ, Redis Streams). Decouples services and survives partial outages.
- Use sync for queries ("show me this user") and async for state changes ("order placed"). Mixing both is normal.

Shared concerns

- **Auth** — a single identity provider (Keycloak, Auth0, your own) issuing JWTs that every service verifies.
- **Tracing** — propagate *traceparent* headers; use OpenTelemetry SDK to instrument FastAPI and outgoing HTTP/DB calls.
- **Schemas** — keep Pydantic models for inter-service contracts in a shared library or generated from OpenAPI.

```
# OpenTelemetry FastAPI instrumentation
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

FastAPIInstrumentor().instrument_app(app)
HTTPXClientInstrumentor().instrument()
SQLAlchemyInstrumentor().instrument(engine=engine)
```

Tip. Resist micro-splitting too early. Start with a well-modularised monolith of routers, and extract a service only when ownership, scaling, or release-cadence boundaries justify the operational overhead.

4.7 CI/CD for FastAPI Apps

A serviceable pipeline runs on every push: lint, type-check, test, build a container, publish it, and deploy. Below is a GitHub Actions workflow that does this for a typical FastAPI project.

```
# .github/workflows/ci.yml
name: ci
on:
  push: { branches: [main] }
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: pg, POSTGRES_DB: test }
        ports: ["5432:5432"]
        options: &gt;-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
      redis:
        image: redis:7
        ports: ["6379:6379"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.13' }
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check .
      - run: mypy app
      - run: pytest -q --cov=app
      - run: pip-audit

  publish:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: |
            ghcr.io/${ github.repository }:${ github.sha }
            ghcr.io/${ github.repository }:latest
```

Deployment options

- **Container platforms** — AWS ECS/Fargate, Google Cloud Run, Azure Container Apps, Fly.io, Render. Cloud Run is the closest thing to zero-ops for a FastAPI image.
- **Kubernetes** — when you already operate it; Helm chart with HPA on request latency or CPU.

- **PaaS** — Heroku, Railway, Render — for very small teams that want one button between commit and live.

■ HANDS-ON PROJECT — Production-ready URL Shortener

Versioned API, Postgres + Redis, Docker image, GitHub Actions CI, OpenTelemetry tracing, generated TS SDK, deployed to a container platform.

Acceptance criteria

- `/v1/shorten` accepts a URL and returns a short code; `/v1/{code}` redirects (HTTP 307).
- Hot codes served from Redis; cold codes from Postgres.
- Per-IP rate limiting at *20/min* on shorten, *200/min* on redirect.
- Multi-stage Dockerfile under 200 MB; non-root user; healthcheck.
- CI lints (ruff), type-checks (mypy), tests (pytest), audits deps (pip-audit), builds image, pushes to GHCR.
- OpenAPI exported and a TypeScript SDK published as a separate artifact.
- Sentry, Prometheus `/metrics`, OpenTelemetry traces to an OTLP collector.

Layout

```
urlshort/
├── app/
│   ├── main.py           # FastAPI app, lifespan, middleware
│   ├── settings.py       # Pydantic Settings
│   ├── deps.py           # db, redis dependencies
│   ├── db.py             # async engine
│   ├── models.py         # Link, Click
│   ├── schemas.py        # Shortenin, ShortenOut
│   └── routers/
│       ├── v1.py         # /v1/shorten, /v1/stats
│       └── redirect.py   # /{code}
├── services/
│   └── shortener.py      # business logic (separate from web layer)
├── alembic/
├── tests/
├── Dockerfile
├── docker-compose.yml
├── .github/workflows/ci.yml
├── scripts/export_openapi.py
└── pyproject.toml
```


Service core (sketch)

```
# app/services/shortener.py
import secrets, string
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from .. import models

ALPHABET = string.ascii_letters + string.digits

def _new_code(n: int = 7) -> str:
    return "".join(secrets.choice(ALPHABET) for _ in range(n))

async def shorten(db: AsyncSession, redis, url: str) -> str:
    for _ in range(5):
        code = _new_code()
        exists = await db.scalar(select(models.Link).where(models.Link.code == code))
        if exists:
            continue
        db.add(models.Link(code=code, url=url))
        await db.commit()
        await redis.set(f"link:{code}", url, ex=3600)
        return code
    raise RuntimeError("could not allocate short code")

async def resolve(db: AsyncSession, redis, code: str) -> str | None:
    cached = await redis.get(f"link:{code}")
    if cached:
        return cached
    link = await db.scalar(select(models.Link).where(models.Link.code == code))
    if link is None:
        return None
    await redis.set(f"link:{code}", link.url, ex=3600)
    return link.url
```

Web layer

```
# app/routers/v1.py
from fastapi import APIRouter, Request
from pydantic import BaseModel, HttpUrl
from ..deps import DB, REDIS, limiter
from ..services import shortener

router = APIRouter(prefix="/v1", tags=["v1"])

class ShortenIn(BaseModel):
    url: HttpUrl

class ShortenOut(BaseModel):
    code: str
    url: HttpUrl

@router.post("/shorten", response_model=ShortenOut)
@limiter.limit("20/minute")
async def shorten(request: Request, payload: ShortenIn, db: DB, redis: REDIS):
    code = await shortener.shorten(db, redis, str(payload.url))
    return ShortenOut(code=code, url=payload.url)

# app/routers/redirect.py
from fastapi import APIRouter, HTTPException, Request, status
from fastapi.responses import RedirectResponse
from ..deps import DB, REDIS, limiter
from ..services import shortener

router = APIRouter()
```

```
@router.get("/{code}")
@limiter.limit("200/minute")
async def go(request: Request, code: str, db: DB, redis: REDIS):
    url = await shortener.resolve(db, redis, code)
    if url is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "unknown code")
    return RedirectResponse(url, status_code=307)
```

From here the path to live is short: *docker build*, *docker push*, *terraform apply* (or your platform's deploy command), point DNS, watch the Prometheus dashboard, fix what is slow. You have shipped a real FastAPI service.

Appendix A — HTTP Status Code Reference

These are the codes you will use day to day in FastAPI services. The full list is in IANA's HTTP status code registry; the ones here are the practical subset.

Code	When to use it
200 OK	Standard success for GET/PUT/PATCH.
201 Created	Resource created; include a Location header.
202 Accepted	Request queued for async processing.
204 No Content	Successful DELETE or PUT with no response body.
301 / 308	Permanent redirect (308 preserves method).
302 / 307	Temporary redirect (307 preserves method).
400 Bad Request	Malformed request the client should fix.
401 Unauthorized	No or invalid credentials; auth required.
403 Forbidden	Authenticated but not allowed.
404 Not Found	Resource does not exist.
409 Conflict	Request conflicts with current state (duplicates, version).
410 Gone	Permanently removed; clients should stop asking.
422 Unprocessable Entity	Validation failed; FastAPI's default validation error.
429 Too Many Requests	Rate limit exceeded.
500 Internal Server Error	Unexpected server error; log it.
502 Bad Gateway	Upstream service failed.
503 Service Unavailable	Overloaded or deliberately offline.
504 Gateway Timeout	Upstream service timed out.

Appendix B — Recommended Project Layout

A layout that scales from a Phase-2 project to a Phase-4 production service:

```
myservice/
  app/
    __init__.py
    main.py           # FastAPI(), lifespan, middleware, router includes
    settings.py       # Pydantic Settings
    db.py             # engine / session factory
    deps.py           # shared dependencies (DB, Redis, current user)
    auth.py           # token issuing/verification, password hashing
    models.py         # SQLAlchemy ORM models
    schemas.py        # Pydantic request/response models
    logging_config.py # structlog setup
    routers/         # one file per domain
      __init__.py
      users.py
      items.py
      health.py       # /health, /ready endpoints
    services/         # pure business logic, no FastAPI imports
      users.py
      items.py
  alembic/
    env.py
    versions/
  tests/
    conftest.py       # fixtures: client, db_session, auth_token
    test_users.py
    test_items.py
  scripts/
    export_openapi.py
  Dockerfile
  docker-compose.yml
  pyproject.toml
  requirements.txt
  requirements-dev.txt
  .env.example
  .github/workflows/ci.yml
```

Note. The cardinal rule: keep services/ free of FastAPI imports. Services know about your domain; routers know about HTTP. This separation makes the business logic trivially unit-testable and reusable in workers or CLI tools.

Appendix C — Cheat Sheet

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install "fastapi[standard]"
fastapi dev main.py
```

Path operation skeleton

```
from typing import Annotated
from fastapi import FastAPI, Path, Query, Depends, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="...", version="1.0.0")

class ItemIn(BaseModel):
    name: str = Field(min_length=1)

class ItemOut(ItemIn):
    id: int

@app.post("/items", response_model=ItemOut, status_code=201, tags=["items"])
def create(payload: ItemIn):
    return ItemOut(id=1, **payload.model_dump())
```

Dependency for DB session

```
def get_db():
    db = SessionLocal()
    try: yield db
    finally: db.close()

DB = Annotated[Session, Depends(get_db)]
```

OAuth2 bearer scheme

```
from fastapi.security import OAuth2PasswordBearer
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
```

Run production

```
gunicorn app.main:app -k uvicorn.workers.UvicornWorker \
    -w 4 -b 0.0.0.0:8000 --timeout 60 --access-logfile -
```

Test

```
from fastapi.testclient import TestClient
from app.main import app
client = TestClient(app)
def test_root():
    assert client.get("/").status_code == 200
```

You are done. This guide has taken you from a first endpoint to a production microservice. The remaining frontier is your own — pick a real problem, ship it, and come back to specific chapters as you need them.

— End of Guide —